Di Simone Andrea Luca

Report sperimentale Algorithm Engineering

Indice

1 Introduzione **3**

2 Background teorico **4**
2.1 Definizioni fondamentali 4
2.2 Classificazione degli algoritmi di massimo flusso 4
2.3 Algoritmi basati su cammini aumentanti 4
2.4 Algoritmi Preflow–Push (Push–Relabel) 4
2.5 Algoritmi con scaling delle capacità 5
2.6 Algoritmi quasi-lineari 5

3 Algoritmo Push-Relabel **6**
3.1 Descrizione generale 6
3.2 Strutture dati 6
3.3 Operazioni fondamentali 6
3.4 Selezione dei nodi attivi: coda FIFO 7
3.5 Euristiche implementate 7
3.5.1 BFS inversa per l'inizializzazione delle altezze 7
3.5.2 Gap heuristic 8
3.5.3 Current-arc (advance pointer) 9
3.6 Complessità 10

4 Algoritmo Capacity Scaling **11**
4.1 Descrizione generale 11
4.2 Strutture dati 11
4.3 Operazioni fondamentali 11
4.4 Struttura a fasi di scaling 12
4.5 Complessità 12
4.6 Gestione di capacità non intere 13
4.7 Confronto con Push-Relabel 14

5 Algoritmo Almost-Linear Time (IPM + Howard) **15**
5.1 Descrizione generale 15
5.2 Riduzione da max-flow a min-cost flow 15
5.3 Funzione potenziale $\Phi(f)$ 15
5.4 Il sottoproblema: minimum ratio cycle 16
5.5 Loop di ottimizzazione interior-point 16
5.6 Ricerca binaria sul valore ottimale 17
5.7 Algoritmo di Howard per il minimum cycle ratio 17
5.8 Strutture dati 18
5.9 Complessità 18
5.10 Gestione di capacità non intere e problemi numerici 19
5.11 Confronto con altri algoritmi 19

6 Dataset e istanze di test **20**
6.1 Istanze random 20
6.2 Reti reali 20
6.3 Istanze con caratteristiche peculiari 20

7 Setup sperimentale **21**
7.1 Hardware e software . . . 21
7.2 Metriche di valutazione . . . 21
7.3 Parametri di configurazione . . . 21

8 Risultati sperimentali **22**
8.1 Confronto sui tempi di esecuzione . . . 22
8.1.1 Almost-Linear Time . . . 22
8.1.2 Capacity Scaling . . . 24
8.1.3 Push-Relabel . . . 28
8.2 Impatto del tipo di capacità . . . 32
8.3 Analisi qualitativa . . . 33

9 Discussione **34**

10 Conclusioni **34**

11 Repository del codice **35**

12 Appendici **35**
12.1 Pseudocodice degli algoritmi . . . 35
12.2 Tabelle complete dei risultati . . . 35
12.3 Parametri dettagliati delle istanze . . . 36
12.4 Grafici aggiuntivi . . . 36
12.5 Riferimenti bibliografici . . . 37

1 Introduzione

Il problema del massimo flusso in una rete costituisce uno dei temi fondamentali nell'ambito dell'ottimizzazione combinatoria e dell'Algorithm Engineering. Una rete di flusso permette di modellare in modo naturale una vasta gamma di problemi reali, che spaziano dal *routing* nelle reti di comunicazione allo *scheduling* di processi, dalla visione artificiale alla progettazione di infrastrutture di trasporto, fino all'analisi di reti energetiche e sociali. Come riportato nel documento originale, "il problema del massimo flusso in una rete trova applicazioni in ambiti quali routing, scheduling, visione artificiale, trasporto e progettazione di reti", evidenziando la centralità di questo paradigma computazionale.

Nonostante la formulazione matematica sia relativamente compatta, il comportamento pratico degli algoritmi di massimo flusso dipende in modo critico da molteplici fattori: la struttura del grafo, la distribuzione delle capacità, la presenza di colli di bottiglia, la densità della rete e l'adozione di specifiche euristiche. Per tali ragioni, l'analisi sperimentale riveste un ruolo essenziale nel valutare l'effettiva efficienza degli algoritmi, andando oltre i soli risultati teorici.

L'obiettivo di questo report è analizzare in profondità il comportamento pratico di diversi algoritmi per il calcolo del massimo flusso, confrontandone prestazioni, robustezza e sensibilità alle caratteristiche delle istanze. Il documento presenta una panoramica teorica degli algoritmi considerati, descrive i dataset utilizzati, illustra la metodologia sperimentale e discute i risultati ottenuti, con particolare attenzione al tempo di esecuzione, al numero di operazioni elementari e all'impatto del tipo di capacità.

Lo studio sperimentale è stato progettato per fornire una valutazione completa e multidimensionale. Sono state considerate istanze random generate secondo diversi modelli (Erdős–Rényi, Barabási–Albert, griglie), reti reali provenienti da SNAP, DIMACS e power grid test cases, nonché istanze con caratteristiche peculiari quali capacità unitarie, capacità floating point e presenza di archi paralleli. Per ogni algoritmo sono state misurate metriche quali il tempo di esecuzione, il numero di operazioni elementari, il numero di fasi di scaling e la stabilità dei risultati su esecuzioni ripetute.

L'analisi combinata di teoria e sperimentazione consente di evidenziare differenze significative tra gli algoritmi, mostrando come alcune tecniche — in particolare le euristiche del metodo Push–Relabel — possano migliorare drasticamente le prestazioni pratiche, mentre altre, come gli algoritmi quasi-lineari, risultano più complessi da implementare e meno competitivi su istanze di dimensione moderata

Il documento presenta una panoramica teorica degli algoritmi considerati, descrive i dataset utilizzati, illustra la metodologia sperimentale e discute i risultati ottenuti, con particolare attenzione al tempo di esecuzione, al numero di operazioni elementari e all'impatto del tipo di capacità.

2 Background teorico

2.1 Definizioni fondamentali

Una rete di flusso è un grafo orientato $G = (V, E)$ dotato di una funzione di capacità $c : E \to \mathbb{R}_{>0}$, una sorgente $s \in V$ e un pozzo $t \in V$. Un flusso è una funzione $f : E \to \mathbb{R}$ che soddisfa:

- **Vincolo di capacità:** $0 \leq f(u, v) \leq c(u, v)$ per ogni arco $(u, v) \in E$.
- **Conservazione del flusso:** per ogni $v \neq s, t$ vale

$$\sum_{u:(u,v)\in E} f(u, v) = \sum_{w:(v,w)\in E} f(v, w).$$

Il valore del flusso è definito come la quantità totale inviata dalla sorgente al pozzo:

$$|f| = \sum_{(s,v)\in E} f(s, v).$$

2.2 Classificazione degli algoritmi di massimo flusso

Gli algoritmi per il calcolo del massimo flusso possono essere suddivisi in diverse famiglie, ciascuna caratterizzata da un differente modello computazionale e da specifiche proprietà teoriche e pratiche.

2.3 Algoritmi basati su cammini aumentanti

Questa famiglia include metodi come Ford–Fulkerson ed Edmonds–Karp. L'idea centrale è individuare ripetutamente un cammino aumentante nella rete residua e inviare flusso lungo tale cammino. La complessità dipende fortemente dalla scelta del cammino.

2.4 Algoritmi Preflow–Push (Push–Relabel)

Gli algoritmi di tipo Preflow–Push, introdotti da Goldberg e Tarjan, mantengono un *preflow* e un'etichetta di altezza per ogni nodo. Le operazioni fondamentali sono:

- **push**: invio di flusso lungo archi ammissibili;
- **relabel**: aumento dell'altezza di un nodo per sbloccare il flusso.

Varianti importanti includono:

- Highest Label,
- FIFO,
- Gap heuristic,
- Global relabeling.

2.5 Algoritmi con scaling delle capacità

Gli algoritmi di tipo *capacity scaling* limitano inizialmente il flusso a capacità elevate e riducono progressivamente la soglia di scaling. Questo approccio riduce il numero di cammini aumentanti "piccoli" e migliora le prestazioni su grafi con capacità molto variabili.

2.6 Algoritmi quasi-lineari

Gli algoritmi quasi-lineari combinano concetti di:

- flussi elettrici,
- solutori Laplaciani,
- metodi di punto interno.

Raggiungono complessità quasi-lineare in teoria, ma richiedono strutture dati avanzate e solutori numerici sofisticati, risultando complessi da implementare e raramente utilizzati in applicazioni pratiche.

3 Algoritmo Push-Relabel

3.1 Descrizione generale

L'algoritmo Push-Relabel, introdotto da Goldberg e Tarjan nel 1988 [6], risolve il problema del flusso massimo su una rete orientata $G = (V, E)$ con capacità $c : E \to \mathbb{R}_{\geq 0}$, sorgente s e pozzo t.

A differenza degli algoritmi basati su cammini aumentanti (Ford-Fulkerson, Edmonds-Karp), Push-Relabel opera *localmente*: mantiene un **pre-flusso**, ovvero una funzione f che può violare la conservazione del flusso nei nodi interni, accumulando un *eccesso* $e(u) = \sum_v f(v, u) - \sum_v f(u, v) \geq 0$. L'algoritmo termina quando tutti gli eccessi nei nodi interni sono stati azzerati.

3.2 Strutture dati

L'implementazione si basa sulle seguenti strutture:

- **Grafo residuo**: per ogni arco (u, v) con capacità $c(u, v)$, si mantengono un arco *forward* con capacità residua $c(u, v) - f(u, v)$ e un arco *reverse* con capacità $f(u, v)$. Ogni arco è rappresentato come una tripla $[to,\ rev_pos,\ res_cap]$, dove rev_pos è l'indice dell'arco inverso nella lista di adiacenza del nodo destinazione.
- **Funzione di altezza** $h : V \to \mathbb{N}$: assegna a ogni nodo un'etichetta intera. Un arco (u, v) è *ammissibile* se e solo se $h(u) = h(v) + 1$ e la sua capacità residua è positiva.
- **Eccesso** $e : V \to \mathbb{R}_{\geq 0}$: quantità di flusso in eccesso accumulata in ogni nodo.
- **Contatore** cnt: array dove $cnt[k]$ tiene il numero di nodi con altezza esattamente k. Utilizzato dall'euristica del gap.

3.3 Operazioni fondamentali

Push. Se un nodo u ha eccesso $e(u) > 0$ e esiste un arco ammissibile (u, v) con capacità residua $r(u, v) > 0$, si invia una quantità di flusso $\delta = \min(e(u),\ r(u, v))$ da u a v:

$$r(u, v) \leftarrow r(u, v) - \delta, \qquad r(v, u) \leftarrow r(v, u) + \delta, \qquad e(u) \leftarrow e(u) - \delta, \qquad e(v) \leftarrow e(v) + \delta.$$

Relabel. Se un nodo u ha eccesso $e(u) > 0$ ma nessun arco ammissibile è disponibile (ovvero tutti gli archi residui uscenti portano a nodi con altezza $\geq h(u)$), si incrementa l'altezza di u:

$$h(u) \leftarrow 1 + \min_{(u,v) \in E_r} h(v),$$

dove E_r è l'insieme degli archi con capacità residua positiva. Questo garantisce che almeno un arco ammissibile diventi disponibile dopo l'operazione.

Inizializzazione. All'avvio si fissa $h(s) = |V|$ e $h(v) = 0$ per ogni $v \neq s$. Tutti gli archi uscenti dalla sorgente s vengono saturati (*pre-saturazione*): per ogni arco (s, v) con $c(s, v) > 0$ si invia un flusso pari all'intera capacità, creando eccesso nei vicini diretti di s.

3.4 Selezione dei nodi attivi: coda FIFO

L'implementazione adotta una **coda FIFO** per la gestione dei nodi attivi (nodi con eccesso positivo, esclusi s e t). Un nodo viene estratto dalla testa della coda, *scaricato* completamente tramite operazioni di push e relabel (*discharge loop*), e reinserito in coda solo se al termine del discharge mantiene ancora eccesso positivo. I nodi che ricevono flusso durante un push e non erano già in coda vengono accodati in fondo.

La selezione FIFO garantisce una complessità nel caso peggiore di $O(V^3)$ [6].

3.5 Euristiche implementate

3.5.1 BFS inversa per l'inizializzazione delle altezze

Motivazione. L'inizializzazione standard pone $h(v) = 0$ per ogni $v \neq s$. Questa è una stima molto approssimativa: i nodi lontani dal pozzo richiedono molti relabel prima che le loro altezze convergano ai valori utili. Ogni relabel costa $O(\deg(u))$ nel caso peggiore, e su grafi grandi o sparsi il numero di relabel necessari prima della convergenza può dominare il tempo di esecuzione.

Descrizione. Prima dell'inizio del ciclo principale, si esegue una **visita BFS sul grafo residuo inverso**, partendo dal pozzo t. Il grafo residuo inverso è costruito considerando solo gli archi con capacità residua positiva e invertendone la direzione: per ogni arco $(u, v) \in E_r$ con $r(u, v) > 0$ si aggiunge l'arco (v, u) al grafo inverso.

La BFS calcola la distanza esatta di ogni nodo da t nel grafo residuo, che costituisce un'altezza ammissibile ottimale al momento dell'inizializzazione:

$$h(v) = d_r(v,\, t), \qquad h(s) = |V|.$$

I nodi irraggiungibili dal pozzo nel grafo residuo vengono inizializzati a $|V| + 1$, rendendoli immediatamente non operativi.

Effetto pratico. Con altezze iniziali già ottimali, il numero totale di operazioni di relabel si riduce drasticamente. Su grafi sparsi con molti nodi, il guadagno è spesso di un ordine di grandezza rispetto all'inizializzazione standard, come confermato dai risultati sperimentali (sezione **??**).

Implementazione.

```
std::vector<int> PushRelabel::_bfs_heights_int(int t) const {
    const int n = _n;
    std::vector<int> h(n, 2 * n); // altezza sentinella per nodi non raggiunti
    h[t] = 0;

    // Costruzione del grafo inverso: radj[v] = nodi u tali che esiste (u→v) con cap > 0
    std::vector<std::vector<int>> radj(n);
    for (int u = 0; u < n; ++u)
        for (auto& e : _adj_int[u])
            if (e.cap > 0)
```

```
                radj[e.to].push_back(u);

    // BFS standard a partire dal pozzo
    std::deque<int> queue;
    queue.push_back(t);
    while (!queue.empty()) {
        int v = queue.front(); queue.pop_front();
        for (int u : radj[v]) {
            if (h[u] == 2 * n) { // nodo non ancora visitato
                h[u] = h[v] + 1;
                queue.push_back(u);
            }
        }
    }
    return h;
}
```

3.5.2 Gap heuristic

Motivazione. Durante l'esecuzione dell'algoritmo, può accadere che nessun nodo abbia altezza pari a un certo valore k con $0 < k < |V|$. In tal caso, i nodi con altezza $h(v) > k$ non possono mai raggiungere il pozzo t (che ha altezza 0) attraverso archi ammissibili, poiché il "gap" nel livello k interrompe qualsiasi cammino verso t. Continuare a processare questi nodi è inutile.

Descrizione. Ogni volta che si esegue un relabel su un nodo u, si decrementa $cnt[h_{\text{old}}]$ (dove h_{old} è l'altezza precedente di u). Se $cnt[h_{\text{old}}] = 0$ e $0 < h_{\text{old}} < |V|$, si è creato un gap al livello h_{old}: tutti i nodi $v \neq s$ con $h_{\text{old}} < h(v) < |V|$ vengono portati all'altezza $|V| + 1$, escludendoli definitivamente dal calcolo:

$$cnt[h_{\text{old}}] = 0 \ \wedge \ 0 < h_{\text{old}} < |V| \implies \forall v \neq s : h_{\text{old}} < h(v) < |V| \Rightarrow h(v) \leftarrow |V| + 1.$$

I current-arc pointer dei nodi coinvolti vengono azzerati.

Effetto pratico. Il gap heuristic elimina intere componenti di nodi irraggiungibili senza doverle esplorare esplicitamente, riducendo in modo significativo il numero totale di operazioni. Sebbene non migliori il bound teorico $O(V^3)$, l'impatto pratico è molto rilevante, specialmente su grafi con struttura a livelli.

Implementazione.

```
if (cnt[old_h] == 0 && old_h > 0 && old_h < n) {
    for (int v = 0; v < n; ++v) {
        if (v != s && h[v] > old_h && h[v] < n) {
            cnt[h[v]]--;
```

```
            h[v] = n + 1;
            cnt[h[v]]++;
            cur[v] = 0; // reimposta current-arc
        }
    }
}
```

3.5.3 Current-arc (advance pointer)

Motivazione. Senza ottimizzazioni, la procedura di discharge di un nodo u scorre ogni volta l'intera lista di adiacenza di u alla ricerca di un arco ammissibile. Se la lista contiene $\deg(u)$ archi, ogni discharge costa $O(\deg(u))$ nel caso peggiore, anche se la maggior parte degli archi non è ammissibile.

Descrizione. Si mantiene per ogni nodo u un **puntatore all'arco corrente** $cur[u]$, inizializzato a 0. Durante il discharge di u:

- Se l'arco $cur[u]$ è ammissibile, si esegue il push e si rimane su quell'arco.
- Se l'arco non è ammissibile, si avanza il puntatore: $cur[u] \leftarrow cur[u] + 1$.
- Se il puntatore raggiunge la fine della lista ($cur[u] = \deg(u)$), tutti gli archi sono stati esaminati senza trovare push ammissibili: si esegue il relabel e si azzera il puntatore, $cur[u] \leftarrow 0$.

Il puntatore non viene mai fatto retrocedere (salvo dopo un relabel), perché dopo un push su (u, v) l'arco rimane ammissibile finché $e(u) > 0$ oppure $r(u, v) = 0$; gli archi precedenti non possono essere diventati ammissibili nel frattempo.

Costo ammortizzato. Ogni arco viene percorso dal puntatore al più una volta per ogni relabel di u. Poiché il numero totale di relabel è $O(V^2)$, il costo totale dei movimenti del puntatore è $O(V^2 \cdot \deg_{\max}) = O(VE)$, ammortizzato su tutta l'esecuzione. Nella pratica, questo riduce drasticamente il numero di esaminazioni inutili degli archi.

Implementazione.

```
// senza current-arc: reset a ogni discharge
    cur[u] = 0;

// durante il discharge
if (cur[u] == static_cast<int>(adj[u].size())) {
    // relabel
    ...
    cur[u] = 0;       // reset dopo relabel
else {
    if (res > 0 && h[u] == h[v] + 1) {
        // Arco ammissibile: invia min(excess[u], res) unità
```

```
        long long delta = std::min(excess[u], res);
        edge.cap                -= delta;
        adj[v][edge.rev].cap    += delta;
        excess[u]               -= delta;
        excess[v]               += delta;
        // Accoda v se diventa attivo
        if (v != s && v != t && !in_queue[v] && excess[v] > 0) {
            queue.push_back(v); in_queue[v] = true;
        }
    } else {
                cur[u]++;
            }
```

3.6 Complessità

Configurazione	Complessità teorica	Note
FIFO puro	$O(V^3)$	Bound teorico, Goldberg & Tarjan 1988
+ Gap heuristic	$O(V^3)$	Stesso bound, ma drasticamente migliore in pratica
+ Current-arc	$O(V^2\sqrt{E})$	Con selezione highest-label; FIFO mantiene $O(V^3)$
+ BFS init	$O(V^3)$	Riduce il numero assoluto di relabel

Table 1: Complessità dell'algoritmo Push-Relabel nelle diverse configurazioni.

4 Algoritmo Capacity Scaling

4.1 Descrizione generale

L'algoritmo Capacity Scaling, introdotto da Gabow nel 1985 [4], risolve il problema del flusso massimo su una rete orientata $G = (V, E)$ con capacità $c : E \to \mathbb{R}_{>0}$, sorgente s e pozzo t.

A differenza dell'algoritmo di Edmonds-Karp, che aumenta il flusso lungo qualsiasi cammino aumentante nella rete residua, Capacity Scaling introduce il concetto di **fase di scaling**: in ogni fase si considera solo la porzione della rete residua in cui gli archi hanno capacità residua almeno pari a una soglia Δ, chiamata Δ-*grafo residuo*. La soglia viene dimezzata progressivamente fino a zero, garantendo che in ogni fase il numero di augmentazioni sia limitato.

4.2 Strutture dati

L'implementazione si basa sulle seguenti strutture:

- **Grafo residuo**: per ogni arco (u, v) con capacità $c(u, v)$, si mantengono un arco *forward* con capacità residua $r(u, v) = c(u, v) - f(u, v)$ e un arco *reverse* con capacità $r(v, u) = f(u, v)$. Ogni arco è rappresentato come una tripla $[to,\ rev_pos,\ res_cap]$.
- **Soglia** Δ: valore che determina quali archi sono ammissibili in una data fase. Solo gli archi con $r(u, v) \geq \Delta$ vengono considerati.
- **Tabella parent**: struttura restituita dalla BFS che codifica il cammino aumentante trovato, nella forma $\{nodo \to (nodo_padre,\ indice_arco)\}$.

4.3 Operazioni fondamentali

BFS nel Δ-grafo residuo. Data la soglia corrente Δ, si esegue una visita BFS sulla rete residua considerando solo gli archi con capacità residua $r(u, v) \geq \Delta$. Se il pozzo t è raggiungibile dalla sorgente s, la BFS restituisce la tabella *parent* che codifica un cammino aumentante; altrimenti restituisce `None` e la fase corrente termina. Il costo è $O(m)$.

Augmentazione. Dato un cammino aumentante codificato in *parent*, si procede in due passi:

1. Si percorre il cammino a ritroso da t a s per trovare il **bottleneck**:

$$\delta = \min_{(u,v) \in P} r(u, v).$$

2. Si percorre nuovamente il cammino a ritroso aggiornando le capacità residue:

$$r(u, v) \leftarrow r(u, v) - \delta, \qquad r(v, u) \leftarrow r(v, u) + \delta.$$

Il flusso inviato δ viene sommato al totale. Il costo è $O(n)$.

4.4 Struttura a fasi di scaling

Inizializzazione di Δ. Si sceglie come valore iniziale di Δ la più grande potenza di 2 non superiore alla massima capacità $U = \max_{(u,v)\in E} c(u,v)$:

$$\Delta_0 = 2^{\lfloor \log_2 U \rfloor}.$$

Nell'implementazione, poiché le capacità possono essere valori reali (irrazionali o razionali), si usa `math.floor` e `math.log2` invece dell'operatore di shift intero:

```
delta = 2 ** math.floor(math.log2(U))
```

Ciclo principale. L'algoritmo procede secondo il seguente schema:

```
while (delta >= EPS) {
    // Fase: aumenta finché esistono cammini nel delta-grafo residuo
    while (true) {
        auto path = _bfs_dbl(s, t, delta);
        if (!path.has_value()) break;
        total += _augment_dbl(s, t, *path);
    }
    delta /= 2.0; // prossima fase con soglia dimezzata
}
return total;
```

Il ciclo esterno scandisce le fasi di scaling: $\Delta_0, \Delta_0/2, \Delta_0/4, \ldots$ Il ciclo interno esegue tutte le augmentazioni possibili nel Δ-grafo residuo corrente. Quando non esistono più cammini aumentanti con capacità residua almeno Δ, si dimezza Δ e si passa alla fase successiva.

Condizione di terminazione. Per capacità intere, l'algoritmo termina quando $\Delta < 1$. Per capacità reali (razionali o irrazionali), si usa una soglia numerica $\varepsilon > 0$ per evitare iterazioni infinite dovute all'aritmetica floating point:

$$\texttt{EPS} = 10^{-12}.$$

Il ciclo prosegue finché $\Delta \geq \varepsilon$.

4.5 Complessità

Numero di fasi. Il numero totale di fasi è $\lfloor \log_2 U \rfloor + 1 = O(\log U)$, dove U è la massima capacità degli archi.

Augmentazioni per fase. In ogni fase con soglia Δ, ogni augmentazione invia almeno Δ unità di flusso. Il flusso massimo è al più $|V| \cdot U$ (somma delle capacità degli archi uscenti da s), quindi il numero di augmentazioni per fase è al più $O(m)$, poiché ogni augmentazione satura almeno un arco nel Δ-grafo residuo e ogni arco può essere saturato al più due volte (nella direzione forward e reverse) prima di uscire dal Δ-grafo.

Costo totale. Ogni augmentazione richiede una BFS di costo $O(m)$ e un'augmentazione di costo $O(n)$. Dominando la BFS, il costo per fase è $O(m \cdot m) = O(m^2)$. Su $O(\log U)$ fasi:

$$T(n, m, U) = O(m^2 \log U).$$

Parametro	**Valore**
Numero di fasi	$O(\log U)$
Augmentazioni per fase	$O(m)$
Costo per augmentazione (BFS)	$O(m)$
Complessità totale	$O(m^2 \log U)$

Table 2: Complessità dell'algoritmo Capacity Scaling.

4.6 Gestione di capacità non intere

L'implementazione estende l'algoritmo originale, concepito per capacità intere, al caso di capacità razionali e irrazionali.

Capacità razionali con scaling. Quando il programma viene lanciato con il flag `-rat`, le capacità vengono convertite in interi esatti prima dell'esecuzione tramite la funzione `scale_rationals`: si calcola il minimo comune multiplo k dei denominatori di tutte le capacità e si moltiplicano tutte le capacità per k. Il flusso risultante viene diviso per k al termine:

$$f_{\text{reale}} = \frac{f_{\text{intero}}}{k}.$$

In questo modo l'algoritmo opera interamente su interi, preservando la correttezza esatta.

Capacità irrazionali e razionali senza scaling. Senza il flag `-rat`, le capacità vengono trattate direttamente come valori `float`. I due adattamenti principali rispetto alla versione intera sono:

- **Calcolo di Δ_0**: si sostituisce l'operatore di shift intero `U.bit_length()` con `math.floor(math.log2(U))`, che funziona correttamente su valori float.
- **Dimezzamento di Δ**: si sostituisce lo shift `delta >>= 1` con la divisione `delta /= 2`, compatibile con float.
- **Condizione di terminazione**: la condizione `delta >= 1` viene sostituita da `delta >= EPS` con `EPS` $= 10^{-12}$, per evitare loop infiniti causati da capacità residue numericamente non nulle ma trascurabili.
- **Soglia nella BFS**: gli archi vengono inclusi nel Δ-grafo residuo se $r(u, v) \geq \Delta$ (confronto diretto tra float).

L'uso di capacità float introduce inevitabilmente errori di arrotondamento. Per capacità irrazionali, il valore del flusso massimo calcolato è un'approssimazione floating point del valore esatto; la precisione è determinata dall'accumulo degli errori durante le augmentazioni.

4.7 Confronto con Push-Relabel

Proprietà	**Capacity Scaling**	**Push-Relabel (FIFO)**
Complessità teorica	$O(m^2 \log U)$	$O(V^3)$
Dipendenza da U	Sì	No
Strategia	Cammini aumentanti	Pre-flusso locale
Capacità reali	Supportate (via EPS)	Supportate (via EPS)
Capacità razionali esatte	Via scaling a interi	Via scaling a interi

Table 3: Confronto tra Capacity Scaling e Push-Relabel.

Capacity Scaling è preferibile quando $m \ll V^2$ (grafi sparsi) e $\log U$ è piccolo. Push-Relabel è preferibile su grafi densi o quando le capacità sono molto grandi.

5 Algoritmo Almost-Linear Time (IPM + Howard)

5.1 Descrizione generale

L'algoritmo Almost-Linear Time, introdotto da Chen, Kyng, Liu, Peng, Gutenberg e Sachdeva nel 2022 [**?**], risolve il problema del massimo flusso e del minimo costo in tempo $m^{1+o(1)}$, dove $m = |E|$ è il numero di archi. La complessità quasi-lineare lo rende asintoticamente superiore agli algoritmi classici (Edmonds-Karp, Push-Relabel, Capacity Scaling) su istanze grandi.

L'algoritmo si basa su due componenti principali:

1. Un **metodo interior-point a riduzione di potenziale** (IPM) che riduce il problema di min-cost flow a una sequenza di $m^{1+o(1)}$ sottoproblemi di *minimum-ratio cycle*.

2. Una **struttura dati dinamica e randomizzata** per risolvere ciascun sottoproblema in tempo ammortizzato $m^{o(1)}$.

Nell'implementazione descritta in questo capitolo si adotta un approccio *statico*: la struttura dati dinamica è sostituita dall'**algoritmo di Howard** [**?**], un oracolo esatto per il minimum cycle ratio (MCR), lasciando invariata la logica dell'IPM. Questo permette di valutare la correttezza e la praticabilità dell'IPM in isolamento, indipendentemente dalla complessità implementativa della struttura dati originale.

5.2 Riduzione da max-flow a min-cost flow

La riduzione standard da max-flow a min-cost flow consiste nell'assegnare costo 0 a tutti gli archi originali e aggiungere un arco virtuale $(t \to s)$ con costo -1 e capacità sufficientemente grande. Un algoritmo di min-cost flow minimizzerà il costo complessivo massimizzando il flusso sull'arco virtuale, che coincide con il valore del massimo flusso s-t.

Formalmente, data un'istanza di max-flow su $G = (V, E)$ con capacità $c : E \to \mathbb{Z}_{>0}$, si costruisce l'istanza di min-cost flow $\hat{G}$ come segue:

- Per ogni arco $e \in E$: costo $c_e = 0$, capacità inferiore $u_e^- = 0$, capacità superiore $u_e^+ = c_e$.
- Arco virtuale $(t \to s)$: costo -1, $u_{ts}^- = 0$, $u_{ts}^+ = \sum_{v:(s,v)\in E} c_{sv}$.
- Costo ottimale noto: $F^* = -f^*$, dove f^* è la stima corrente del flusso massimo.

5.3 Funzione potenziale $\Phi(f)$

Il cuore dell'IPM è la funzione potenziale a riduzione di potenziale:

$$\Phi(f) = 20m \log\big(c^\top f - F^*\big) + \sum_{e\in E} \Big[\big(u_e^+ - f_e\big)^{-\alpha} + \big(f_e - u_e^-\big)^{-\alpha}\Big]$$

dove $\alpha = 1/\log_2(1000 \cdot m \cdot U)$, $U = \max_e |u_e^+|$, e F^* è il costo ottimale (assunto noto dalla ricerca binaria, cfr. Sezione 5.6).

Il potenziale combina:

- **Obiettivo**: il termine $20m \log(c^\top f - F^*)$ decresce al diminuire del costo del flusso.

- **Barriera**: la somma $\sum_e (\cdot)^{-\alpha}$ penalizza le soluzioni prossime ai bordi del dominio ammissibile $u^- < f < u^+$, impedendo la violazione dei vincoli di capacità. La barriera di potenza $x^{-\alpha}$ con $0 < \alpha \ll 1$ è meno restrittiva della barriera logaritmica standard $-\log x$: lascia avvicinare il flusso ai limiti di capacità fino a valori estremamente prossimi a zero prima di divergere, garantendo maggiore precisione sulla soluzione finale.

Gradienti e lunghezze. Il gradiente di Φ rispetto a f_e è:

$$g(f)_e = \nabla_e \Phi(f) = 20m\,(c^\top f - F^*)^{-1} c_e + \alpha\big(u_e^+ - f_e\big)^{-1-\alpha} - \alpha\big(f_e - u_e^-\big)^{-1-\alpha}$$

La lunghezza di un arco misura quanto il flusso è vincolato dalle capacità:

$$\ell(f)_e = \big(u_e^+ - f_e\big)^{-1-\alpha} + \big(f_e - u_e^-\big)^{-1-\alpha}$$

5.4 Il sottoproblema: minimum ratio cycle

Ad ogni iterazione dell'IPM occorre trovare una circolazione $\Delta \in \mathbb{R}^E$ (con $B^\top \Delta = 0$) che minimizzi il rapporto:

$$\min_{B^\top \Delta = 0} \frac{g^\top \Delta}{\|L\Delta\|_1}$$

dove $L = \mathrm{diag}(\ell)$. Il ciclo di minimo rapporto fornisce la direzione di discesa ottimale per $\Phi(f)$: il numeratore negativo indica che muoversi lungo Δ riduce il potenziale, mentre il denominatore cresce all'avvicinarsi ai bordi del dominio, limitando l'ampiezza del passo.

Nell'implementazione, il sottoproblema è risolto dall'**algoritmo di Howard** (cfr. Sezione 5.7), che opera sul grafo non orientato trattando ogni arco $e = (u, v)$ come percorribile in entrambe le direzioni: se percorso in direzione opposta, il gradiente dell'arco cambia segno ($-g_e$), così come la componente di Δ.

5.5 Loop di ottimizzazione interior-point

Flusso ammissibile iniziale. L'IPM richiede un punto di partenza strettamente interno al dominio ammissibile di Φ. Si costruisce il flusso centrale $f_0 = (u^- + u^+)/2$, che soddisfa per costruzione i vincoli di capacità ma in generale viola la conservazione del flusso ($B^\top f_0 \neq 0$).

Per correggere gli squilibri nodali $\hat{d} = B^\top f_0$, si aggiunge un nodo fittizio v^* collegato a ciascun nodo sbilanciato:

- Se $\hat{d}_v > 0$ (eccesso su v): arco $v^* \to v$ con flusso iniziale $\hat{d}_v$ e capacità superiore $2\hat{d}_v$.
- Se $\hat{d}_v < 0$ (deficit su v): arco $v \to v^*$ con flusso iniziale $|\hat{d}_v|$ e capacità superiore $2|\hat{d}_v|$.

Il costo degli archi ausiliari è $c = 4mU^2$, sufficientemente elevato da renderli non attraenti per l'ottimizzatore, così da non influire sulla soluzione dell'istanza originale. Il flusso iniziale f_0 esteso alla nuova istanza soddisfa la conservazione del flusso e appartiene all'interno del dominio ammissibile di Φ.

Ciclo principale. Il loop di ottimizzazione è strutturato come segue:

```
while c·f - F* >= threshold:
    g = calc_gradients(f)
    l = calc_lengths(f)
    (min_ratio, Delta) = howard(I, g, l)
    assert min_ratio < 0
    eta = -(kappa^2) / (50 * g · Delta)
    f = f + eta * upscale * Delta
    assert Phi(f_new) < Phi(f_old)
```

Lo **step-size** η è calcolato come:

$$\eta = -\frac{\kappa^2}{50 \cdot g^\top \Delta}$$

con $\kappa \in (0, 1)$ fattore di qualità dell'approssimazione. Poiché $g^\top \Delta < 0$, risulta $\eta > 0$. Il fattore `upscale` (tipicamente 10^3) amplifica il passo permettendo una convergenza più rapida, a scapito di una maggiore esposizione al rischio di violazione dei vincoli su istanze grandi.

Le invarianti verificate ad ogni iterazione sono:

1. Conservazione del flusso: $\|B^\top f\|_\infty < 10^{-10}$.
2. Ratio negativo: min_ratio < 0.
3. Decrescita stretta del potenziale: $\Phi(f_{\text{new}}) < \Phi(f_{\text{old}})$.

Il loop termina quando $c^\top f - F^* < \varepsilon$ (con $\varepsilon = 10^{-5}$) oppure quando $\Phi(f)$ non è più finito (bordo del dominio raggiunto per errori numerici).

5.6 Ricerca binaria sul valore ottimale

La funzione potenziale $\Phi(f)$ richiede la conoscenza del costo ottimale F^*. Come osservato in [**?**], questo valore non è noto a priori: si stima tramite ricerca binaria nell'intervallo $[0, U_s]$, dove $U_s = \sum_{v:(s,v)\in E} c_{sv}$ è la somma delle capacità uscenti dalla sorgente.

Ad ogni iterazione binaria si testa un candidato mid:

- Se il flusso restituito da `max_flow_with_guess(mid)` è $<$ mid: il valore non è raggiungibile, si abbassa `high`.
- Altrimenti: il valore è raggiungibile, si alza `low`.

La ricerca termina in $O(\log U_s)$ iterazioni, ciascuna delle quali invoca il loop IPM descritto nella Sezione 5.5.

5.7 Algoritmo di Howard per il minimum cycle ratio

L'algoritmo di Howard [**? ?**] è un metodo iterativo per il problema del minimum cycle ratio (MCR) su grafi pesati diretti (o non diretti). Dato un grafo $G = (V, E)$ con pesi g_e (gradiente) e $\ell_e > 0$ (lunghezza) per ogni arco, il MCR è definito come:

$$\lambda^* = \min_{\text{ciclo } C} \frac{\sum_{e\in C} g_e}{\sum_{e\in C} \ell_e}$$

Schema dell'algoritmo. Howard costruisce e raffina iterativamente una *policy*: un sottografo π in cui ogni nodo ha esattamente un arco uscente. Su tale policy si esegue il rilevamento di cicli; per ogni ciclo trovato si calcola il ratio e si aggiorna la policy scegliendo, per ciascun nodo, l'arco che minimizza localmente il ratio. Il processo converge al ciclo di ratio minimo.

Per supportare il grafo non orientato dell'IPM, l'implementazione considera ogni arco $e = (u, v)$ anche in direzione inversa (v, u): il gradiente dell'arco percorso in senso inverso è $-g_e$, mentre la lunghezza rimane ℓ_e (sempre positiva). Il risultato di Howard è un vettore $\Delta \in \{-1, 0, +1\}^E$, dove $\Delta_e = +1$ se l'arco è percorso in avanti, $\Delta_e = -1$ se percorso all'indietro e $\Delta_e = 0$ se non appartiene al ciclo.

5.8 Strutture dati

- `MinCostFlow`: struttura centrale che incapsula il grafo, i vettori Eigen c, u^-, u^+, la matrice di incidenza $B \in \mathbb{R}^{m \times n}$, i parametri U e α, e i metodi `phi`, `calc_gradients`, `calc_lengths`.
- **Matrice di incidenza** B: $B_{e,v} = +1$ se l'arco e esce da v, $B_{e,v} = -1$ se entra in v, 0 altrimenti. Permette di esprimere la conservazione del flusso come $B^\top f = 0$ e di calcolare la divergenza nodale in $O(m)$ con una moltiplicazione matrice-vettore.
- **Vettori Eigen (`VectorXd`)**: tutti i flussi, gradienti e lunghezze sono rappresentati come vettori `float64` di Eigen, che supporta nativamente prodotti scalari, norme e operazioni entrywise utilizzate nel calcolo di Φ, g e ℓ.
- **Lista di adiacenza** `adj`: per ogni nodo v, `adj[v]` contiene gli indici degli archi incidenti (entranti e uscenti), usata da Howard per la visita dei vicini.
- **Mappa** `undirected_edge_to_indices`: hash map da coppia di nodi (a, b) agli indici degli archi tra a e b, con lookup $O(1)$ ammortizzato (hash basato su codifica a 64 bit).

5.9 Complessità

Numero di iterazioni IPM. [CKLPGS22] dimostra che, con un oracolo MCR che restituisce un ciclo $m^{o(1)}$-approssimato e $\kappa \geq m^{-o(1)}$, il potenziale $\Phi(f)$ decresce di $\Omega(\kappa^2)$ per iterazione. Partendo da $\Phi(f^{(0)}) \leq 200m \log(mU)$ e terminando a $\Phi(f) \leq -200m \log(mU)$, il numero totale di iterazioni è $O(m\kappa^{-2}) = m^{1+o(1)}$.

Costo per iterazione. Con la struttura dati dinamica di [CKLPGS22], ogni iterazione costa $m^{o(1)}$ ammortizzato, portando a un costo totale $m^{1+o(1)}$. Nell'implementazione statica qui descritta, Howard ha complessità per iterazione $O(nm)$ nel caso peggiore, rendendo la complessità pratica dell'implementazione significativamente più alta.

Ricerca binaria. La ricerca binaria aggiunge un fattore $O(\log U)$, portando la complessità totale teorica a $m^{1+o(1)} \log U$ per il max-flow.

Componente	Complessità
Iterazioni IPM	$m^{1+o(1)}$
Oracolo MCR (Howard, statico)	$O(nm)$ per iterazione
Ricerca binaria	$O(\log U)$ invocazioni IPM
Complessità totale (teorica, con DS)	$m^{1+o(1)} \log U$
Complessità totale (implementazione statica)	$O(nm^2 \log U)$

Table 4: Complessità dell'algoritmo Almost-Linear Time con oracolo statico (Howard).

5.10 Gestione di capacità non intere e problemi numerici

L'implementazione usa `long long` per le capacità degli archi (evitando overflow su istanze con capacità elevate) e `float64` (Eigen `VectorXd`) per i flussi e le quantità dell'IPM. Questa scelta diverge dal modello di calcolo a virgola fissa di [CKLPGS22], che garantisce la rappresentazione di tutti i valori intermedi in $O(\log^{O(1)} z)$ bit (con z dimensione dell'input in bit).

I principali problemi numerici riscontrati sono:

- **Overflow sul costo ausiliario**: il termine $4mU^2$ nei flussi ammissibili può eccedere `long long` per valori grandi di m e U.
- **Arrotondamento**: la soluzione approssimata dell'IPM viene arrotondata a interi al termine. Se il flusso è distribuito su più archi in modo quasi-uniforme (ad es. 0.5 su ciascuno di due archi), l'arrotondamento può violare la conservazione del flusso sui singoli archi pur restituendo il valore totale corretto.
- **Divergenza di Φ**: quando il flusso si avvicina al bordo del dominio ammissibile, la barriera di potenza può divergere a $+\infty$ per effetti di arrotondamento in virgola mobile, causando la terminazione anticipata del loop.

5.11 Confronto con altri algoritmi

Algoritmo	Complessità	Dipende da U	Praticità
Edmonds-Karp	$O(m^2 n)$	No	Alta
Capacity Scaling	$O(m^2 \log U)$	Sì	Alta
Push-Relabel (FIFO)	$O(n^3)$	No	Alta
Almost-Linear (IPM)	$m^{1+o(1)} \log U$	Sì	Bassa (costanti grandi)

Table 5: Confronto tra l'algoritmo Almost-Linear Time e algoritmi classici per il max-flow.

Come osservato sperimentalmente in [**?**], nonostante la complessità asintotica ottimale, l'IPM richiede circa 10^4–10^5 iterazioni su istanze con $m \leq 500$, contro le poche centinaia di aggiornamenti di Edmonds-Karp o Push-Relabel sulle stesse istanze. Le costanti nascoste nell'esponente $o(1)$ rendono l'algoritmo poco competitivo su istanze di dimensione pratica, confermando che il salto teorico non si traduce in un vantaggio pratico immediato.

6 Dataset e istanze di test

6.1 Istanze random

Sono stati generati grafi casuali utilizzando diversi modelli:

- Erdős–Rényi $G(n, p)$,
- Barabási–Albert (scale-free),
- griglie bidimensionali e tridimensionali.

Le capacità sono state assegnate secondo distribuzioni uniformi o gaussiane, sia intere che reali, con range variabile a seconda dell'esperimento.

6.2 Reti reali

Sono state utilizzate reti provenienti da:

- SNAP,
- DIMACS,
- power grid test cases,
- road networks.

Ogni rete è stata preprocessata per:

- rimuovere nodi isolati,
- normalizzare le capacità,
- convertire il formato in DIMACS.

6.3 Istanze con caratteristiche peculiari

Sono state incluse istanze con:

- capacità unitarie,
- capacità intere piccole,
- capacità intere molto grandi,
- capacità floating point,
- archi paralleli o strutture irregolari.

Queste istanze permettono di valutare la sensibilità degli algoritmi alla natura delle capacità e alla struttura del grafo.

7 Setup sperimentale

7.1 Hardware e software

Gli esperimenti sono stati eseguiti su una macchina con le seguenti caratteristiche:

- **CPU:** *Ryzen 5 7600X 5.3 GHz*
- **RAM:** *32 GB DDR5 6000 MHz*
- **Sistema operativo:** *Windows 11*
- **Compilatore:** *MinGW*
- **Librerie utilizzate:** *Eigen*

7.2 Metriche di valutazione

Le metriche considerate includono:

- tempo di esecuzione,

7.3 Parametri di configurazione

Per ogni algoritmo sono stati specificati:

- heuristiche abilitate,
- soglie di scaling,
- precisione numerica per capacità reali.

8 Risultati sperimentali

8.1 Confronto sui tempi di esecuzione

In questa sezione vengono riportati i tempi di esecuzione dei diversi algoritmi su tutte le istanze considerate. I risultati mostrano differenze significative a seconda della struttura del grafo, della densità e del tipo di capacità. Sono stati prodotti grafici e tabelle per evidenziare:

- l'andamento del tempo di esecuzione al variare della dimensione del grafo;
- la sensibilità alle capacità intere, reali e unitarie.

Queste misure permettono di correlare il comportamento empirico con la complessità teorica e di individuare eventuali colli di bottiglia.

8.1.1 Almost-Linear Time

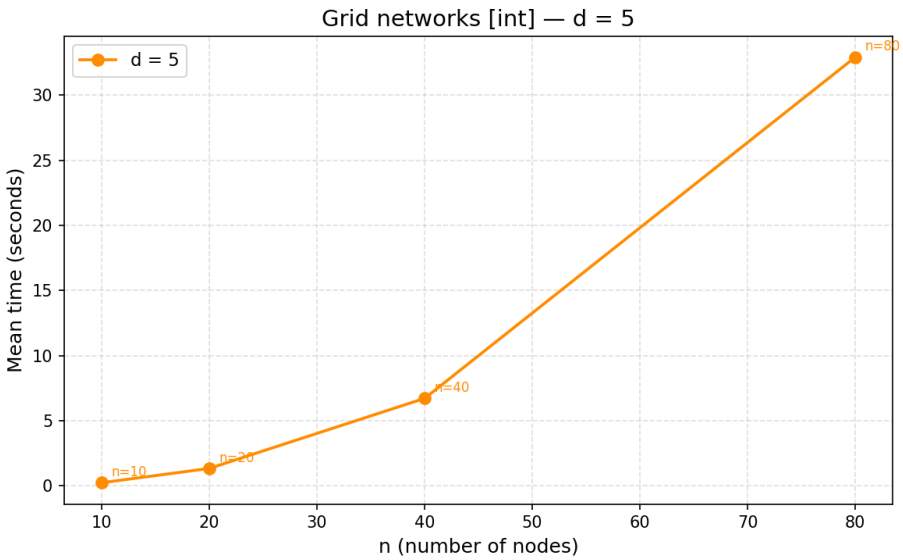

Figure 1: Almost-Linear Time — Grid, capacità intere, $d = 5$

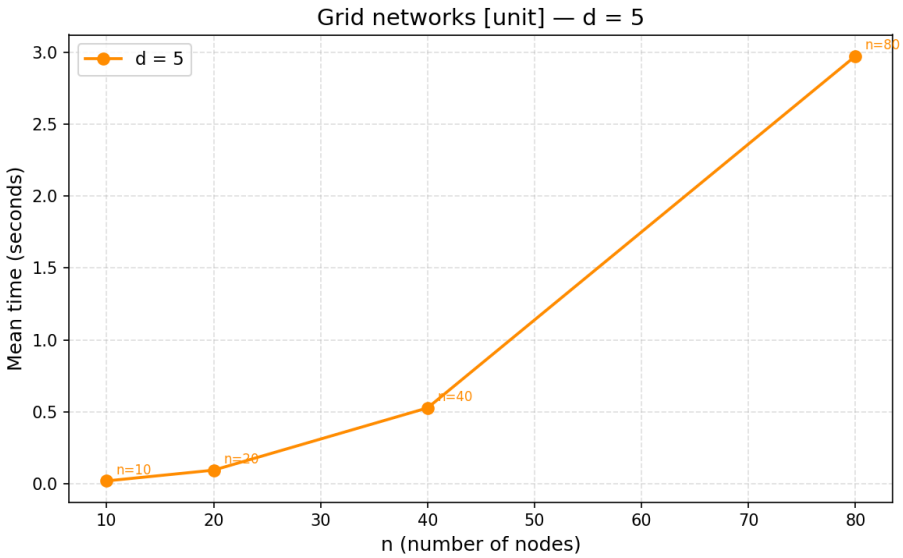

Figure 2: Almost-Linear Time — Grid, capacità unitarie, $d = 5$

Grafo Grid

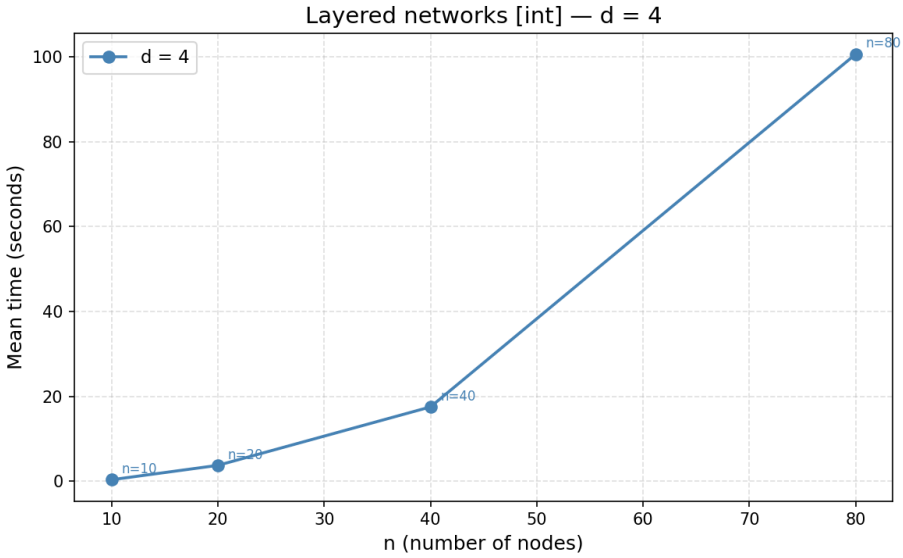

Figure 3: Almost-Linear Time — Layered, capacità intere, $d = 4$

Grafo Layered

8.1.2 Capacity Scaling

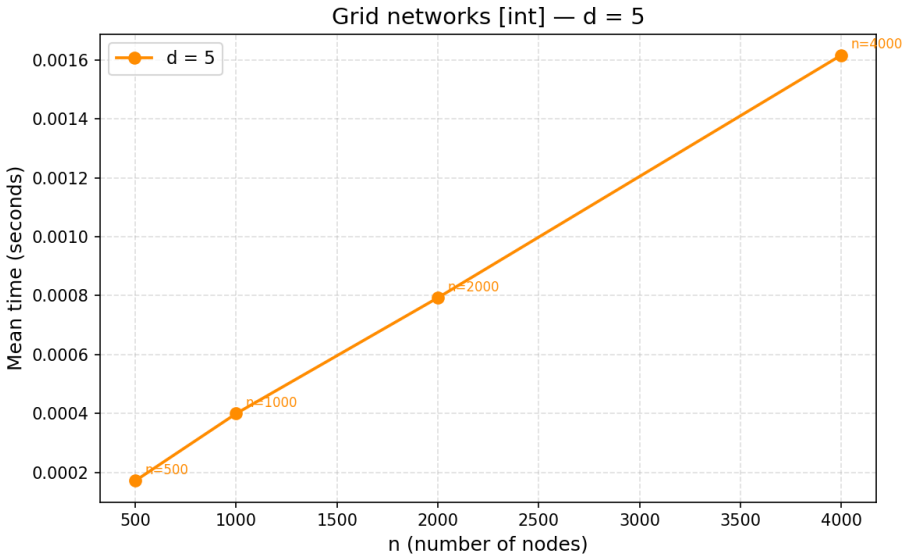

Figure 4: Capacity Scaling — Grid, capacità intere, $d = 5$

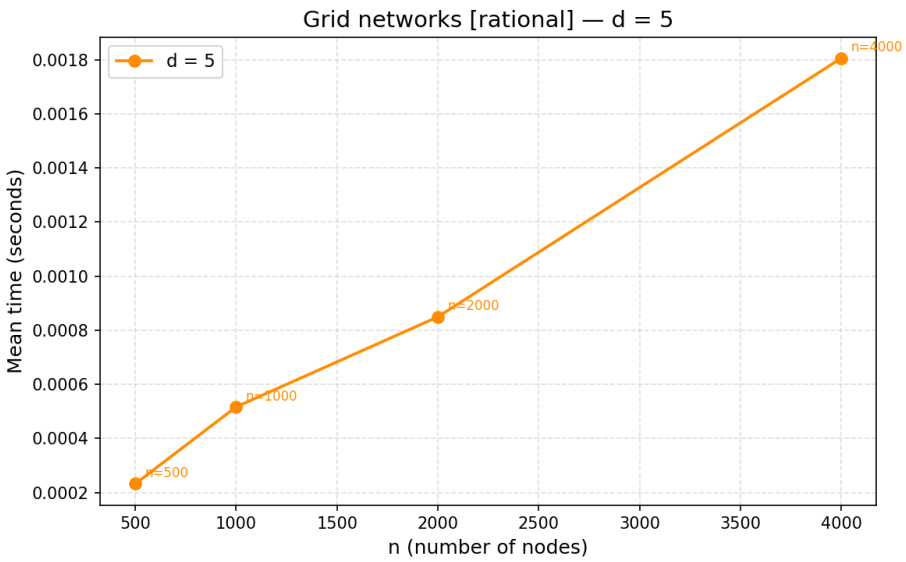

Figure 5: Capacity Scaling — Grid, capacità razionali, $d = 5$

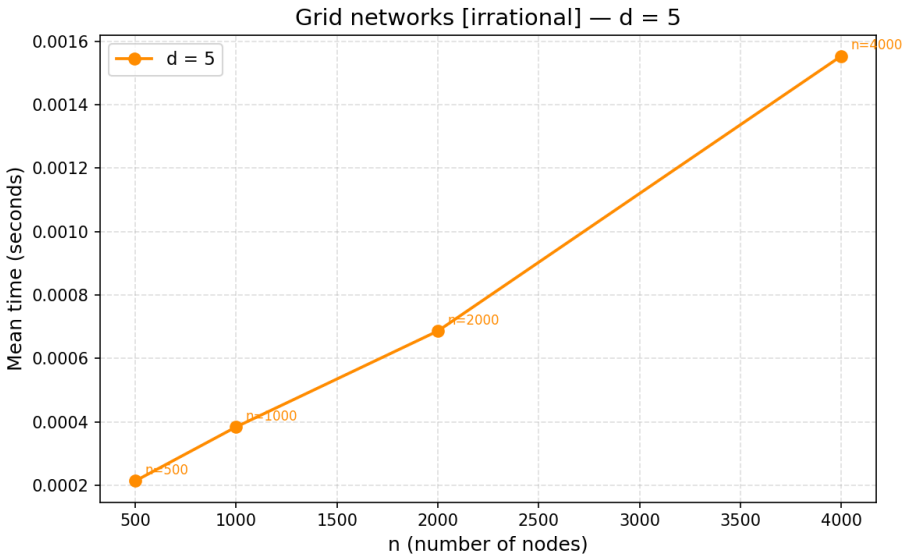

Figure 6: Capacity Scaling — Grid, capacità irrazionali, $d = 5$

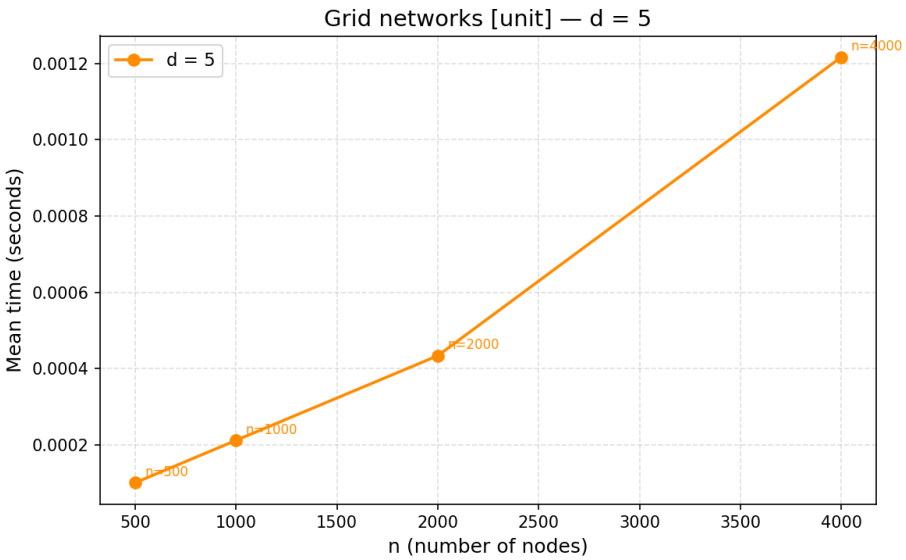

Figure 7: Capacity Scaling — Grid, capacità unitarie, $d = 5$

Grafo Grid

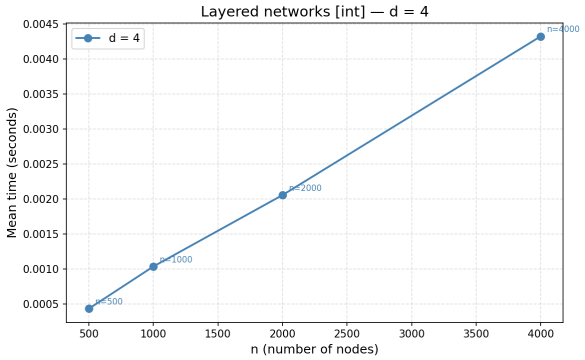

Figure 8: CS — Layered, int, $d = 4$

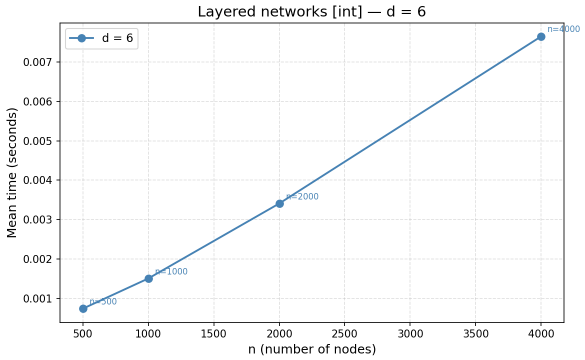

Figure 9: CS — Layered, int, $d = 6$

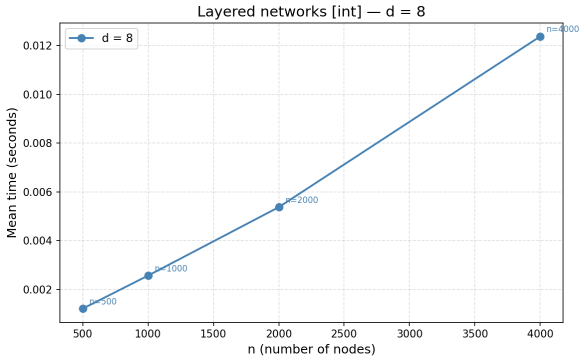

Figure 10: CS — Layered, int, $d = 8$

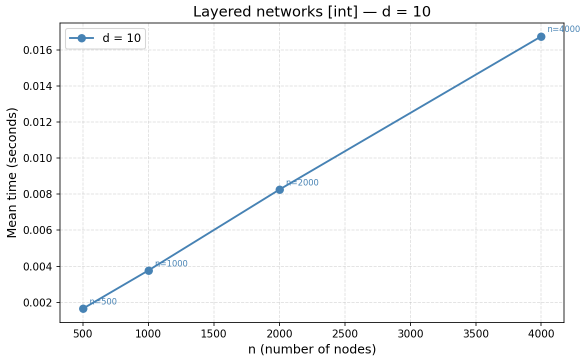

Figure 11: CS — Layered, int, $d = 10$

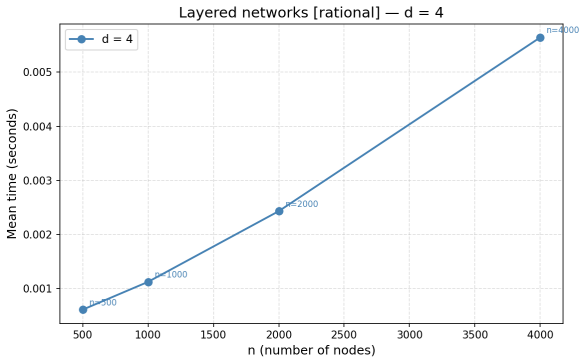

Figure 12: CS — Layered, razionali, $d = 4$

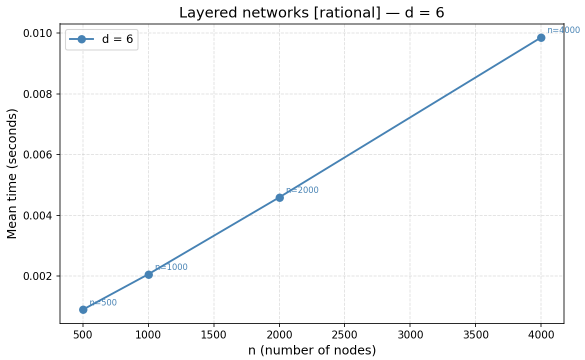

Figure 13: CS — Layered, razionali, $d = 6$

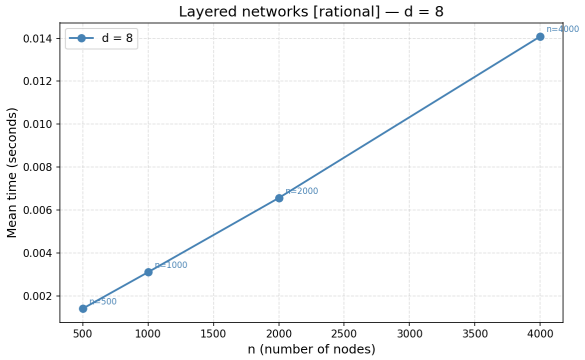

Figure 14: CS — Layered, razionali, $d = 8$

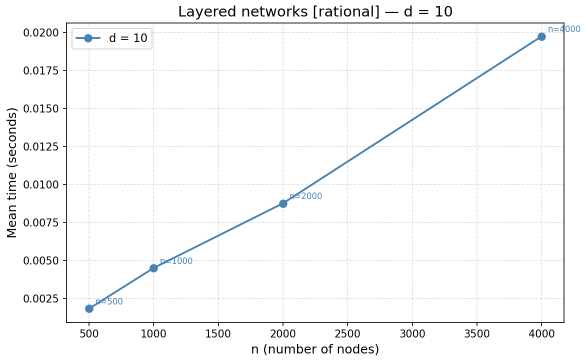

Figure 15: CS — Layered, razionali, $d = 10$

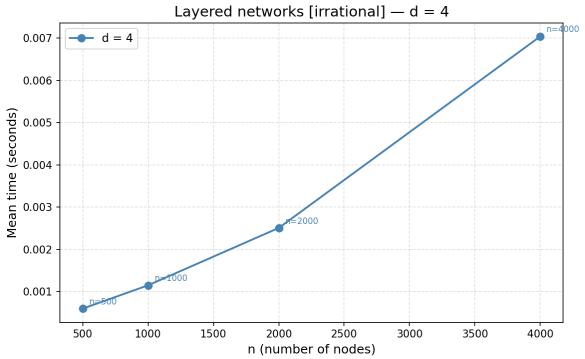

Figure 16: CS — Layered, irrazionali, $d = 4$

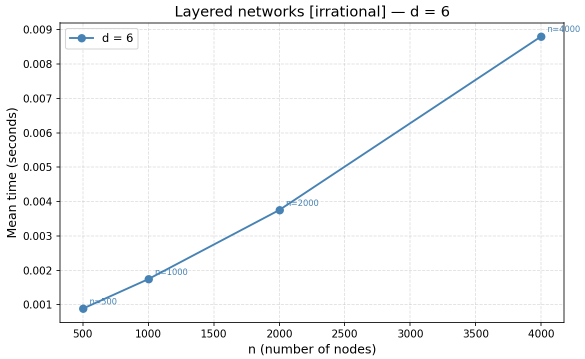

Figure 17: CS — Layered, irrazionali, $d = 6$

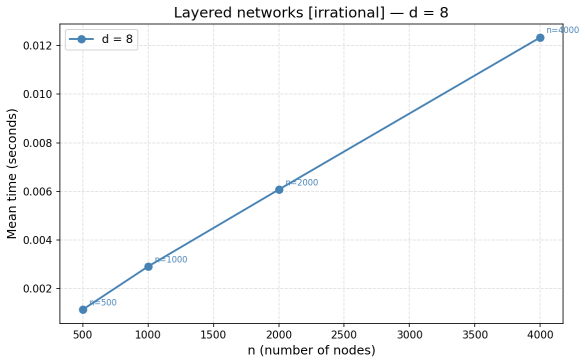

Figure 18: CS — Layered, irrazionali, $d = 8$

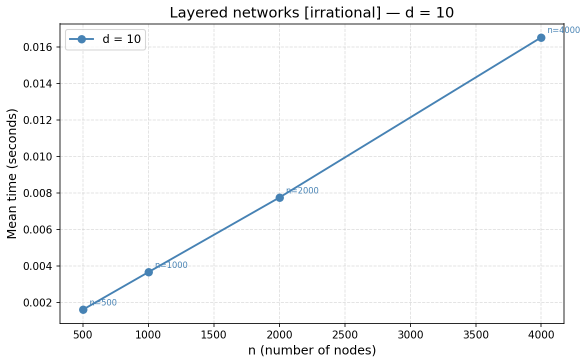

Figure 19: CS — Layered, irrazionali, $d = 10$

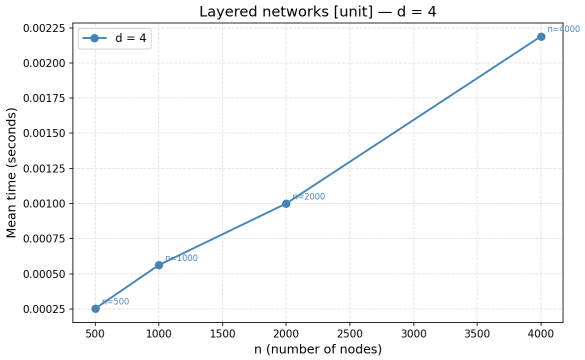

Figure 20: CS — Layered, unitarie, $d = 4$

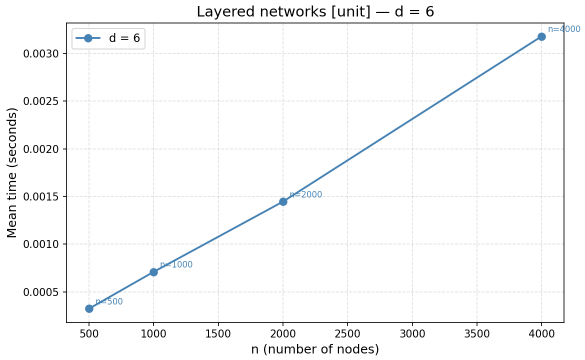

Figure 21: CS — Layered, unitarie, $d = 6$

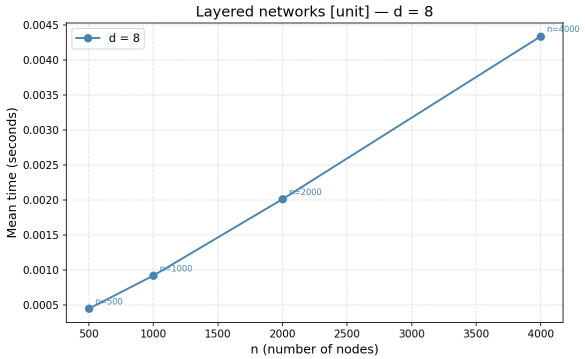

Figure 22: CS — Layered, unitarie, $d = 8$

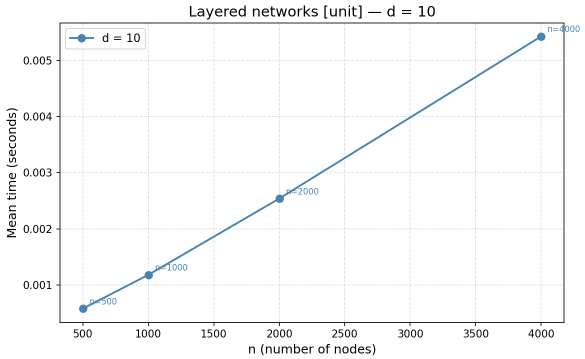

Figure 23: CS — Layered, unitarie, $d = 10$

Grafo Layered

8.1.3 Push-Relabel

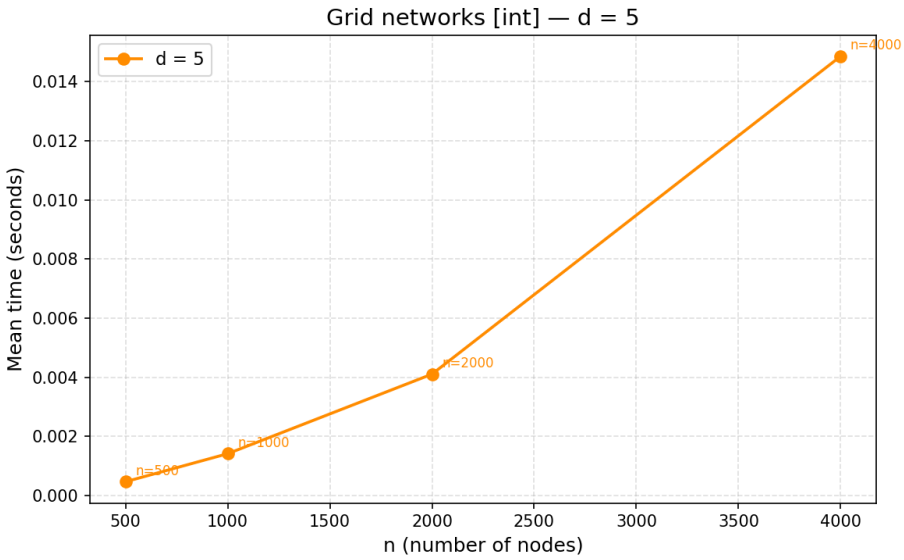

Figure 24: Push-Relabel — Grid, capacità intere, $d = 5$

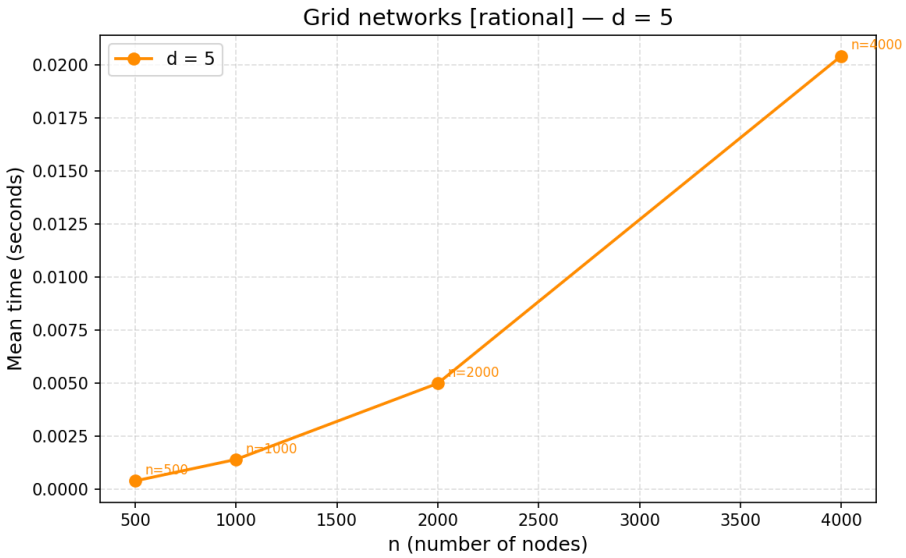

Figure 25: Push-Relabel — Grid, capacità razionali, $d = 5$

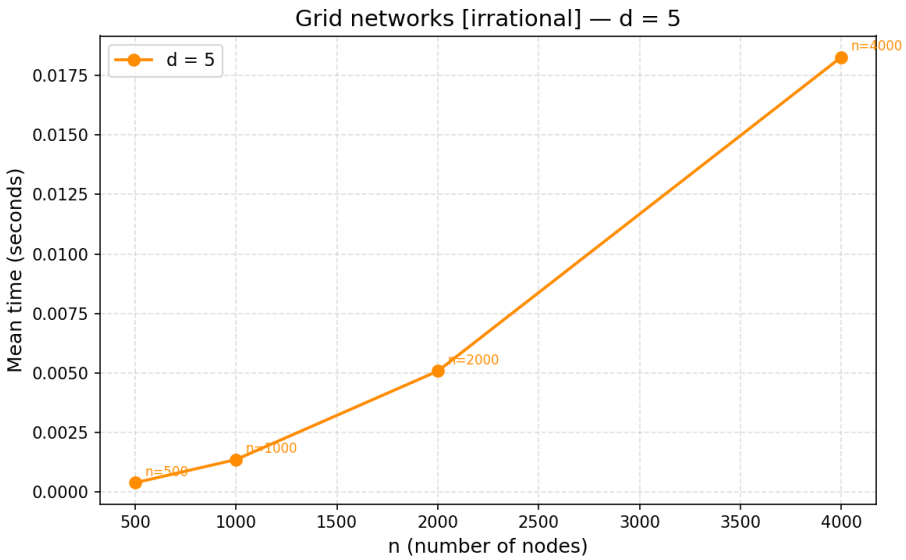

Figure 26: Push-Relabel — Grid, capacità irrazionali, $d = 5$

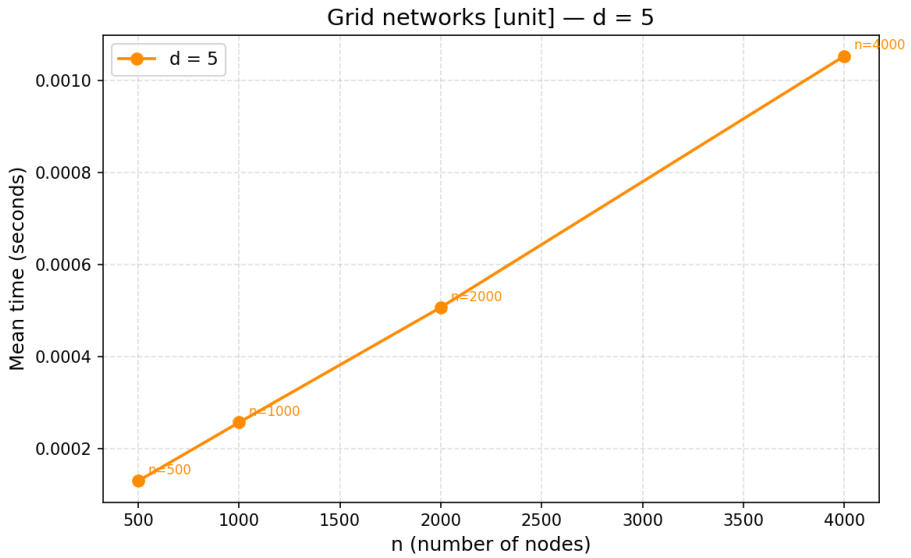

Figure 27: Push-Relabel — Grid, capacità unitarie, $d = 5$

Grafo Grid

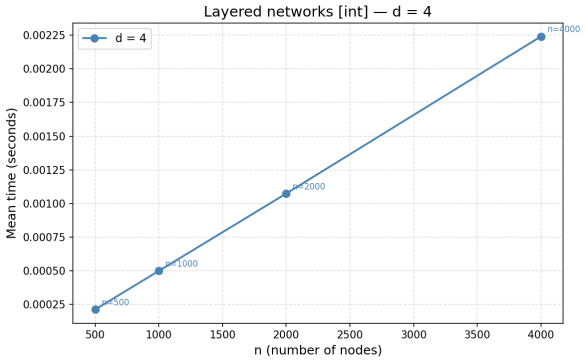

Figure 28: PR — Layered, int, $d = 4$

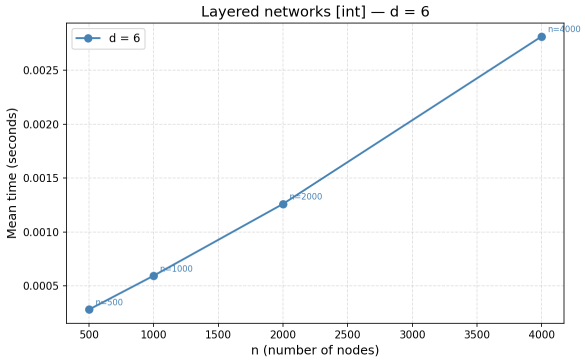

Figure 29: PR — Layered, int, $d = 6$

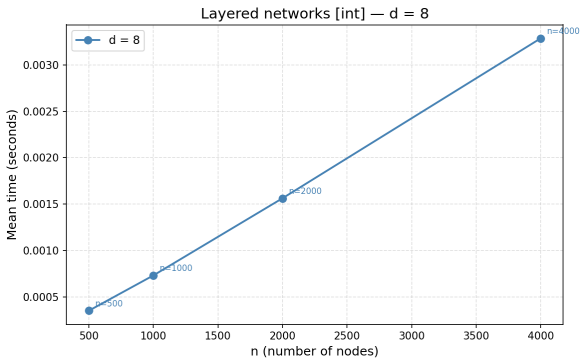

Figure 30: PR — Layered, int, $d = 8$

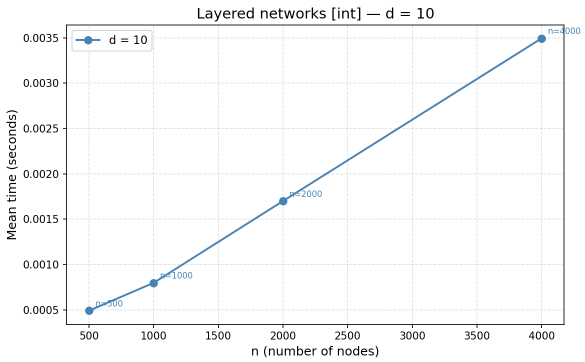

Figure 31: PR — Layered, int, $d = 10$

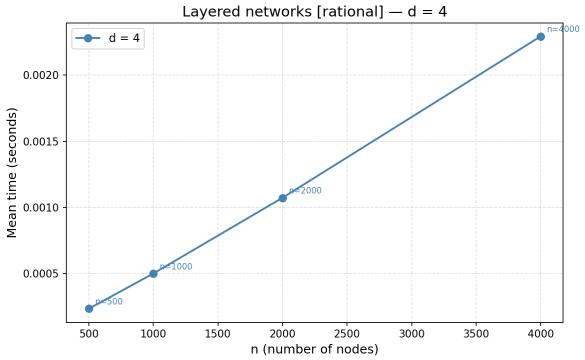

Figure 32: PR — Layered, razionali, $d = 4$

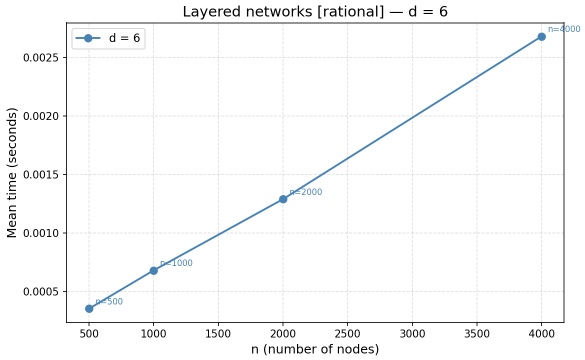

Figure 33: PR — Layered, razionali, $d = 6$

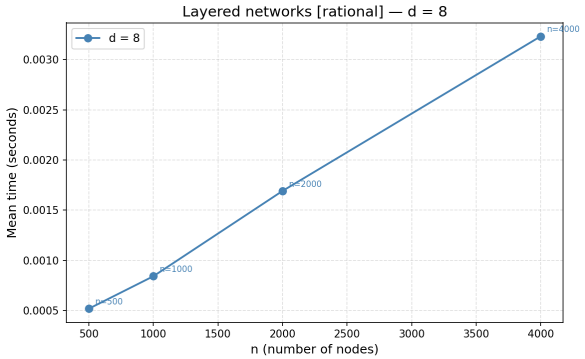

Figure 34: PR — Layered, razionali, $d = 8$

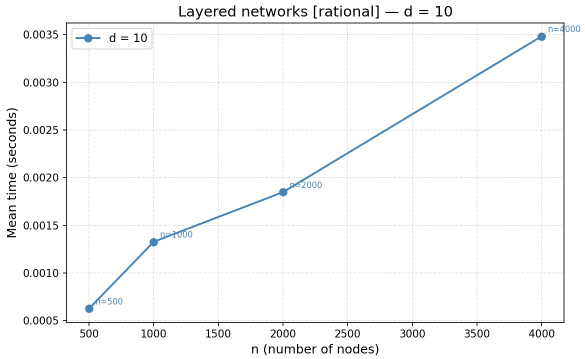

Figure 35: PR — Layered, razionali, $d = 10$

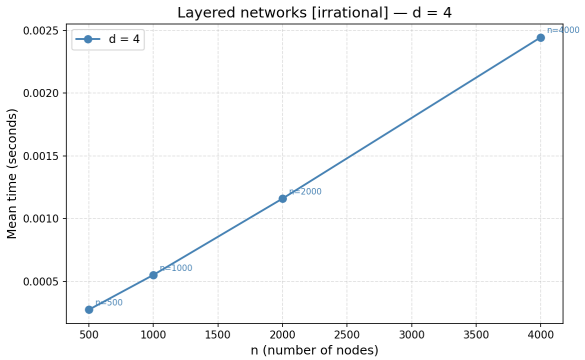

Figure 36: PR — Layered, irrazionali, $d = 4$

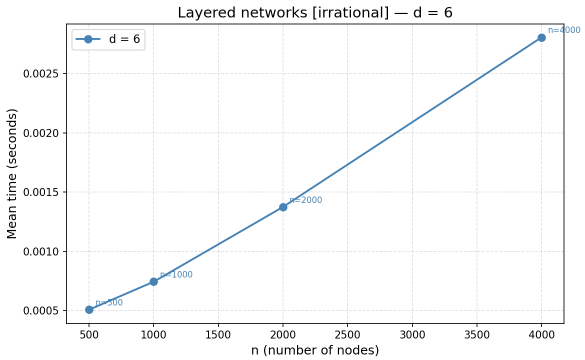

Figure 37: PR — Layered, irrazionali, $d = 6$

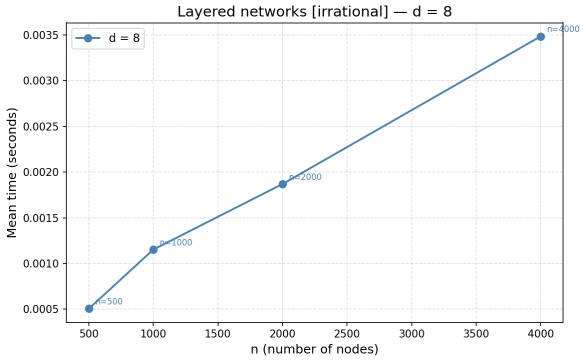

Figure 38: PR — Layered, irrazionali, $d = 8$

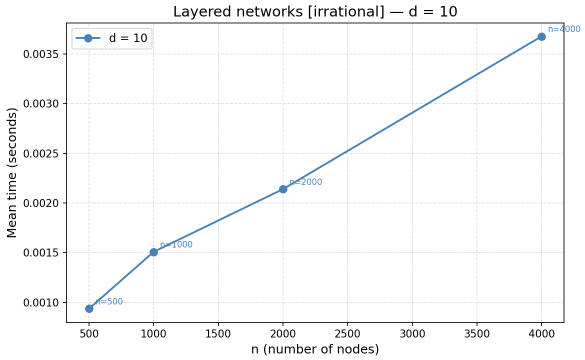

Figure 39: PR — Layered, irrazionali, $d = 10$

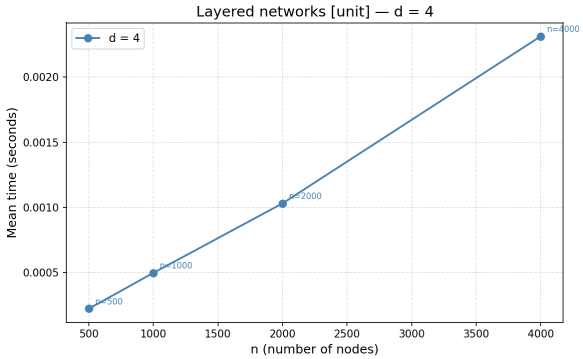

Figure 40: PR — Layered, unitarie, $d = 4$

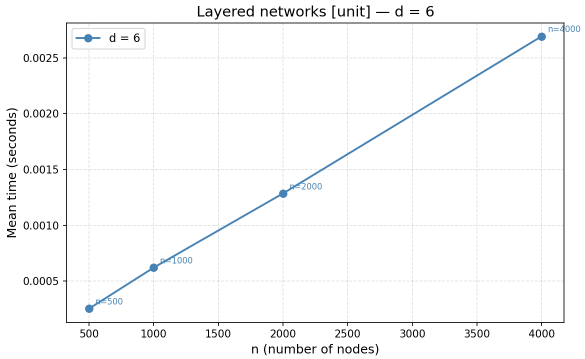

Figure 41: PR — Layered, unitarie, $d = 6$

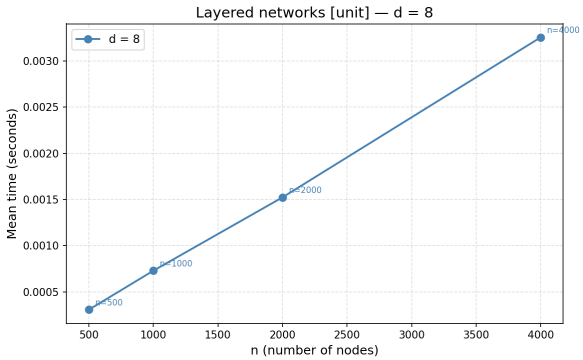

Figure 42: PR — Layered, unitarie, $d = 8$

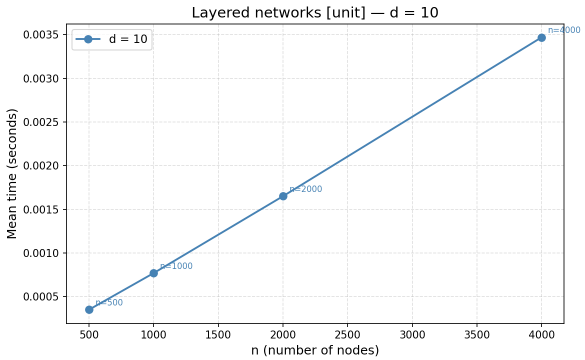

Figure 43: PR — Layered, unitarie, $d = 10$

Grafo Layered

8.2 Impatto del tipo di capacità

Sono stati analizzati tre scenari principali:

- capacità intere nel range [1,1000];
- capacità frazionali numeratore nel range [1,1000] e denominatore nel range [1,20];
- capacità reali (floating point);
- capacità unitarie

8.3 Analisi qualitativa

Oltre ai risultati quantitativi, è stata condotta un'analisi qualitativa considerando:

- robustezza rispetto a variazioni della struttura del grafo;
- sensibilità alla presenza di archi paralleli o colli di bottiglia;

Questa analisi fornisce una visione più completa del comportamento pratico degli algoritmi.

9 Discussione

I risultati sperimentali evidenziano come il comportamento pratico degli algoritmi di massimo flusso dipenda fortemente dalla struttura del grafo e dal tipo di capacità.

In particolare:

- **Push–Relabel** risulta generalmente il più competitivo su una vasta gamma di istanze, grazie alla sua capacità di sfruttare efficacemente le heuristiche disponibili.
- **Capacity scaling** offre miglioramenti significativi quando il range delle capacità è ampio, riducendo il numero di cammini aumentanti "piccoli" e migliorando la stabilità delle prestazioni.
- Gli **algoritmi quasi-lineari**, pur essendo teoricamente molto efficienti, richiedono strutture dati avanzate e solutori numerici complessi, rendendoli difficili da applicare in contesti pratici e meno competitivi su istanze di dimensione moderata.

Un aspetto rilevante emerso dall'analisi è che la complessità teorica non sempre riflette accuratamente il comportamento reale, soprattutto in presenza di grafi irregolari, capacità floating point o strutture con colli di bottiglia.

La combinazione di analisi teorica e sperimentale si rivela quindi essenziale per comprendere appieno le prestazioni degli algoritmi di massimo flusso.

10 Conclusioni

Il presente studio ha analizzato il comportamento pratico di diversi algoritmi per il calcolo del massimo flusso, confrontandone prestazioni, robustezza e sensibilità alle caratteristiche delle istanze.

I risultati mostrano che:

- push relabel è un algoritmo universalmente migliore;
- la scelta dell'algoritmo dipende dalla struttura del grafo e dal tipo di capacità;
- Push–Relabel rappresenta spesso la scelta più solida in contesti generali;
- gli algoritmi con scaling risultano particolarmente efficaci quando il range delle capacità è ampio;
- gli algoritmi quasi-lineari, pur interessanti dal punto di vista teorico, presentano limitazioni pratiche significative.

Il lavoro evidenzia l'importanza di un approccio sperimentale rigoroso per valutare le prestazioni reali degli algoritmi di massimo flusso e suggerisce possibili estensioni future, tra cui:

- analisi su istanze di dimensione ancora maggiore;
- studio dell'impatto di tecniche di parallelizzazione;
- integrazione di nuove heuristiche per migliorare la scalabilità.

11 Repository del codice

Il codice utilizzato per generare gli esperimenti, raccogliere i risultati e produrre le visualizzazioni è disponibile nel seguente repository:

[https://github.com/Di-Simone-dev/AE-MaxFlow]

Il repository è organizzato come segue:

- /**src**: implementazioni degli algoritmi di massimo flusso (Push–Relabel, Capacity Scaling, varianti, ecc.);
- /**datasets**: script per la generazione delle istanze random e per la conversione delle reti reali in formato DIMACS;
- /**experiments**: script per l'esecuzione automatica dei test e la raccolta delle metriche;
- /**plots**: notebook e script per la produzione dei grafici utilizzati nel report;
- /**results**: tabelle, log e statistiche ottenute durante l'esecuzione degli esperimenti.

Il repository include inoltre istruzioni dettagliate per la riproduzione degli esperimenti, comprese le dipendenze software e i parametri utilizzati.

12 Appendici

12.1 Pseudocodice degli algoritmi

In questa appendice vengono riportati i principali pseudocodici utilizzati nel report, inclusi:

- algoritmo Push–Relabel (versione base e varianti);
- algoritmo di Capacity Scaling;
- procedure ausiliarie per la gestione della rete residua.

Gli pseudocodici sono presentati in forma compatta per facilitare la consultazione.

12.2 Tabelle complete dei risultati

Per completezza, si riportano le tabelle contenenti tutti i risultati sperimentali raccolti, incluse:

- tempi di esecuzione per ogni istanza;
- numero di operazioni elementari;
- memoria utilizzata;
- statistiche aggregate (medie, deviazioni standard, percentili).

12.3 Parametri dettagliati delle istanze

Questa sezione include:

- dimensioni dei grafi (numero di nodi e archi);
- distribuzioni delle capacità;
- parametri di generazione per le istanze random;
- caratteristiche strutturali delle reti reali.

12.4 Grafici aggiuntivi

Sono riportati ulteriori grafici non inclusi nel corpo principale del report, tra cui:

- boxplot dei tempi di esecuzione;
- grafici log-log per istanze di grandi dimensioni;
- confronti tra varianti dello stesso algoritmo.

Questi grafici forniscono una visione più approfondita del comportamento degli algoritmi analizzati.

12.5 Riferimenti bibliografici

References

[1] Goldberg, A. V., & Tarjan, R. E. (1988). A new approach to the maximum-flow problem. *Journal of the ACM*, 35(4), 921–940. https://doi.org/10.1145/48014.61051

[2] Cheriyan, J., & Maheshwari, S. N. (1989). Analysis of preflow push algorithms for maximum network flow. *SIAM Journal on Computing*, 18(6), 1057–1086.

[3] Hochbaum, D. S. (2008). The pseudoflow algorithm: A new algorithm for the maximum-flow problem. *Operations Research*, 56(4), 992–1009.

[4] Gabow, H. N. (1985). Scaling algorithms for network problems. *Journal of Computer and System Sciences*, 31(2), 148–168. https://doi.org/10.1016/0022-0000(85)90039-X

[5] Ahuja, R. K., Magnanti, T. L., & Orlin, J. B. (1993). *Network Flows: Theory, Algorithms, and Applications*. Prentice Hall.

[6] Goldberg, A. V., & Tarjan, R. E. (1988). A new approach to the maximum-flow problem. *Journal of the ACM*, 35(4), 921–940. https://doi.org/10.1145/48014.61051